



Pontifícia
Universidade
Católica do
Rio de Janeiro

DEI
DEPARTAMENTO
DE ENGENHARIA
INDUSTRIAL



ENG 4560 – Projeto Integrado VI: Distribuição Física | 2026.1

Aula 11: Metaheurísticas – Iterated Local Search (ILS)

Prof. Marcello Congro

marcellocongro@puc-rio.br



CTC
CENTRO TÉCNICO CIENTÍFICO / PUC-Rio

Onde estamos no curso?

- Construímos uma trajetória metodológica completa e progressiva para o problema de roteirização da Prolog. Na Sprint 1, formulamos matematicamente o CVRP e o resolvemos com um solver de otimização inteira — obtemos soluções ótimas garantidas, mas o tempo computacional cresce exponencialmente com o número de clientes.
- Na Sprint 2, avançamos para as **heurísticas**: primeiro construímos soluções viáveis rapidamente com o Nearest Neighbor e o Clarke-Wright, depois as refinamos com busca local (2-opt e Relocate). O resultado foi bom: soluções de qualidade em segundos, mesmo para instâncias maiores. Mas ao final da Sprint 2, chegamos a um limite conhecido: um ótimo local.
- A busca local encontra uma solução que nenhum dos seus movimentos consegue melhorar — e para. Não porque encontrou a melhor solução possível, mas porque ficou presa num vale local do espaço de busca.



Como fazer a busca local escapar dos ótimos locais que ela mesma cria?

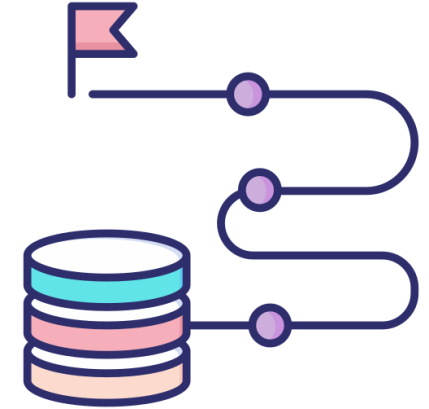
→ *Iterated Local Search (ILS)!*



<https://prologtransportes.com.br/>

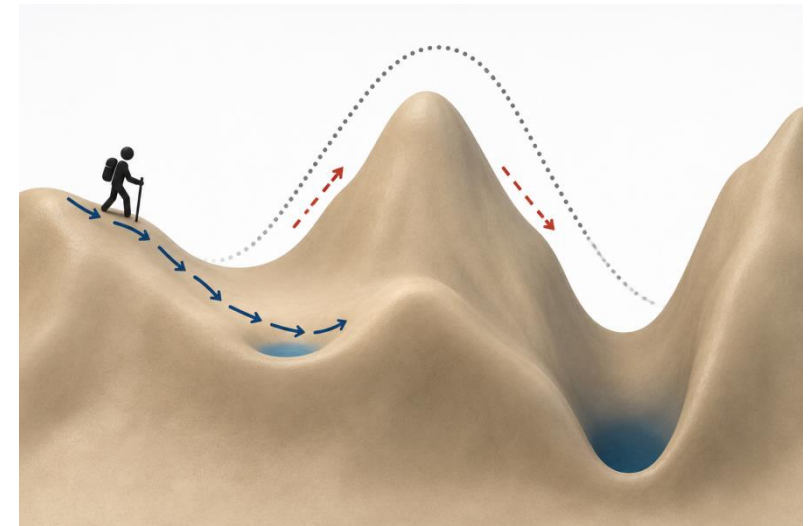
O que fizemos na Sprint #2

- A Sprint 2 entregou um pipeline de resolução em dois estágios:
 - ❖ Heurística construtiva para gerar uma solução inicial viável, seguida de busca local para refiná-la;
 - ❖ Isso produziu planos de entrega para a empresa com menor custo, menor distância e melhor aproveitamento da frota do que qualquer solução inicial.
- Os movimentos de busca local que usamos 2-opt (reorganiza a sequência de visita dentro de uma rota) e Relocate (move clientes entre rotas) são eficazes porque cada um deles reduz sistematicamente o custo em cada aplicação. Eles param somente quando nenhum movimento adicional consegue melhorar.
- Esse "parar quando não há mais melhoria" é exatamente o critério que define um ótimo local: uma solução S tal que $\text{custo}(S) \leq \text{custo}(S')$ para todo S' na vizinhança de S . A solução é localmente ótima em relação à vizinhança definida pelos movimentos usados.
 - ❖ O problema: ótimo local não é ótimo global.
 - ❖ Para instâncias pequenas ($C1$, $C2$), a diferença pode ser pequena.
 - ❖ Para instâncias maiores ($C3$, $C4$) e para o problema real da Prolog com dezenas de clientes e restrições operacionais, essa diferença pode ser significativa em termos de custo e número de veículos.



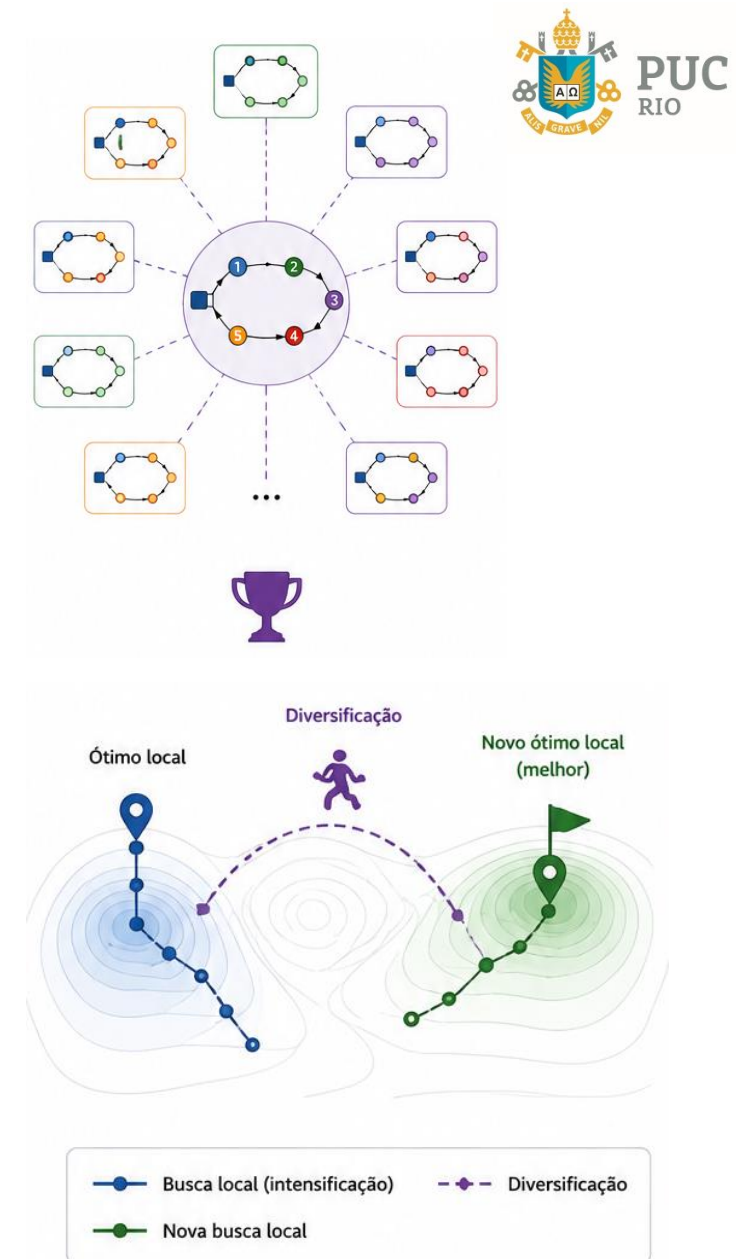
O problema do ótimo local

- A busca local é equivalente a um caminhante que sempre desce ladeira abaixo: a cada passo, ele move para o vizinho de menor altitude. Esse processo é eficiente, mas tem uma consequência inevitável: o caminhante para quando atinge qualquer vale, mesmo que esse vale seja raso e existam vales muito mais profundos em outras regiões da paisagem.
- O motivo é estrutural: a busca local só enxerga a vizinhança imediata. Para chegar a um vale mais profundo que está do outro lado de uma colina, o caminhante precisaria subir temporariamente
 - ❖ A busca local nunca sobe, porque nunca aceita movimentos que aumentam o custo.



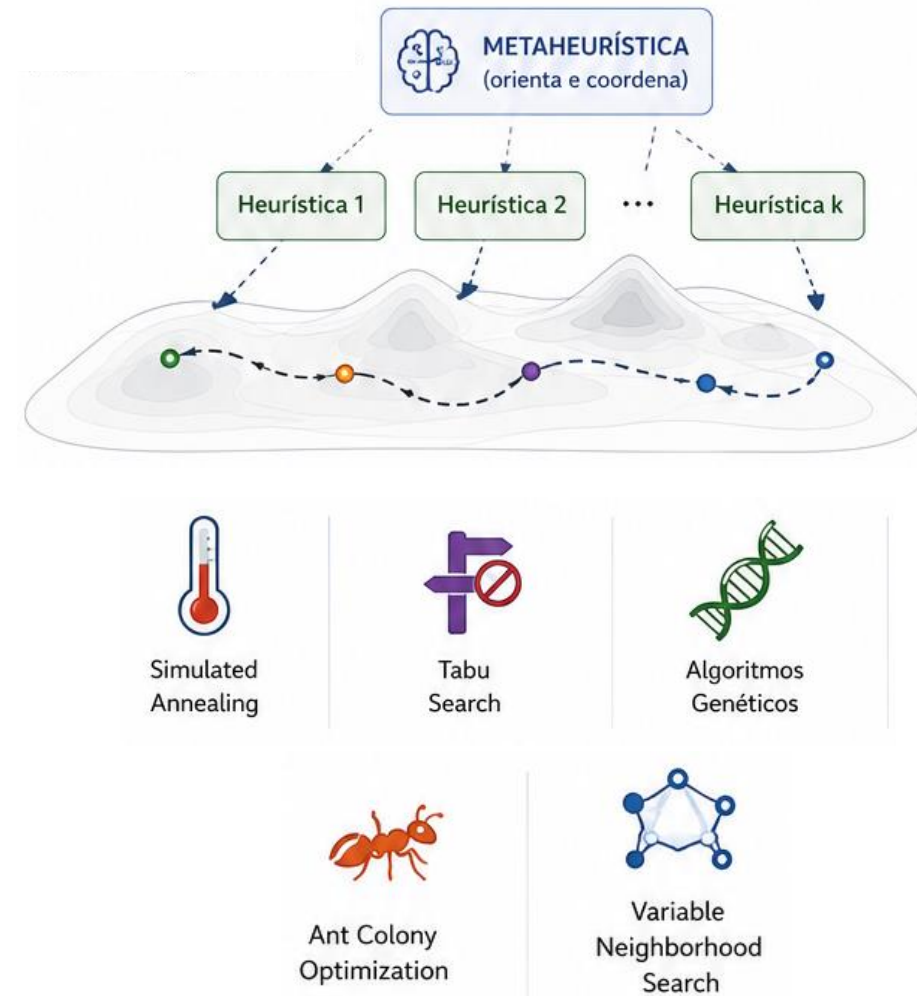
Por que a busca local para onde para?

- Uma mesma solução pode ser ótimo local para movimentos 2-opt e não ser ótimo local para movimentos Relocate
 - ❖ Por isso, combinar múltiplos movimentos na busca local (como fizemos na Sprint 2) amplia a vizinhança e encontra ótimos locais de melhor qualidade.
- Mas há um limite para essa estratégia: aumentar a vizinhança tem custo computacional quadrático ou cúbico
- Para instâncias grandes, há um ponto onde expandir a vizinhança não compensa mais: o tempo computacional explode antes de o algoritmo convergir
 - Instâncias C3 e C4: isso já era perceptível.
- Portanto, a solução não está em expandir indefinidamente a vizinhança da busca local. Está em uma estratégia diferente: escapar do ótimo local por outro mecanismo, preservando a busca local como ela está, mas adicionando uma fase de **diversificação** que a leva para regiões inexploradas do espaço de soluções.



A ideia de metaheurística

- Uma metaheurística é um arcabouço algorítmico de alto nível que orienta e coordena heurísticas mais simples para explorar o espaço de soluções de forma mais eficaz do que cada heurística individualmente conseguiria. O prefixo "meta" vem do grego e significa "além de" → uma metaheurística está além da heurística, dirigindo-a.
- A ideia central das metaheurísticas modernas é equilibrar dois mecanismos complementares e antagônicos: intensificação e diversificação.
 - ❖ A busca local pura é pura intensificação, pois ela explora profundamente a região onde está, até esgotar as possibilidades. O que falta é diversificação: mecanismos para explorar *novas regiões* do espaço de soluções.
- Existem muitas metaheurísticas: Simulated Annealing, Tabu Search, Algoritmos Genéticos, Ant Colony Optimization, Variable Neighborhood Search.
- Nesta sprint, focaremos em uma única metaheurística, aplicada profundamente ao caso prático: **o Iterated Local Search**.



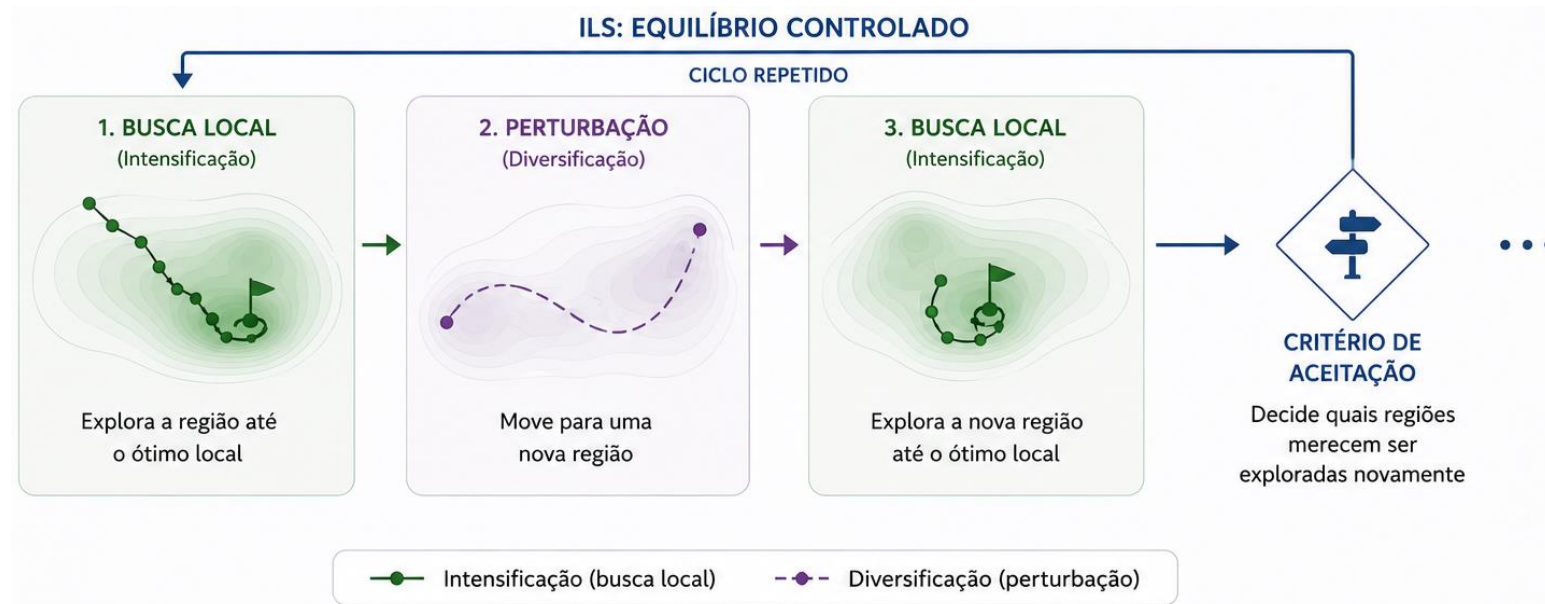
Intensificação e diversificação

- **Intensificação:** é o mecanismo de explorar profundamente uma região promissora do espaço de soluções.
 - ❖ Ex.: busca local. Dado um ponto de partida, ela extrai o máximo de melhoria possível naquela vizinhança antes de parar. Toda boa metaheurística precisa de um componente de intensificação forte.
- **Diversificação:** é o mecanismo de explorar regiões novas e potencialmente mais promissoras do espaço de soluções.
 - ❖ Sem diversificação, o algoritmo converge cedo para um ótimo local e desperdiça a maior parte do tempo computacional disponível. A diversificação é o que impede o algoritmo de ficar preso.



Intensificação e diversificação (cont.)

- O equilíbrio entre os dois é o problema central do design de metaheurísticas. Intensificação demais produz convergência prematura: o algoritmo fica preso no primeiro ótimo local razoável que encontra. Diversificação demais produz busca aleatória: o algoritmo pula de região em região sem explorar nenhuma profundamente, e o custo raramente diminui de forma sustentada.
- No ILS, esse equilíbrio é controlado por dois componentes: a perturbação (diversificação) e a busca local (intensificação). A perturbação move o algoritmo para uma nova região; a busca local explora essa região até o ótimo local.



O ciclo repetido cria um processo de busca equilibrado, e o critério de aceitação decide quais regiões merecem ser exploradas novamente.

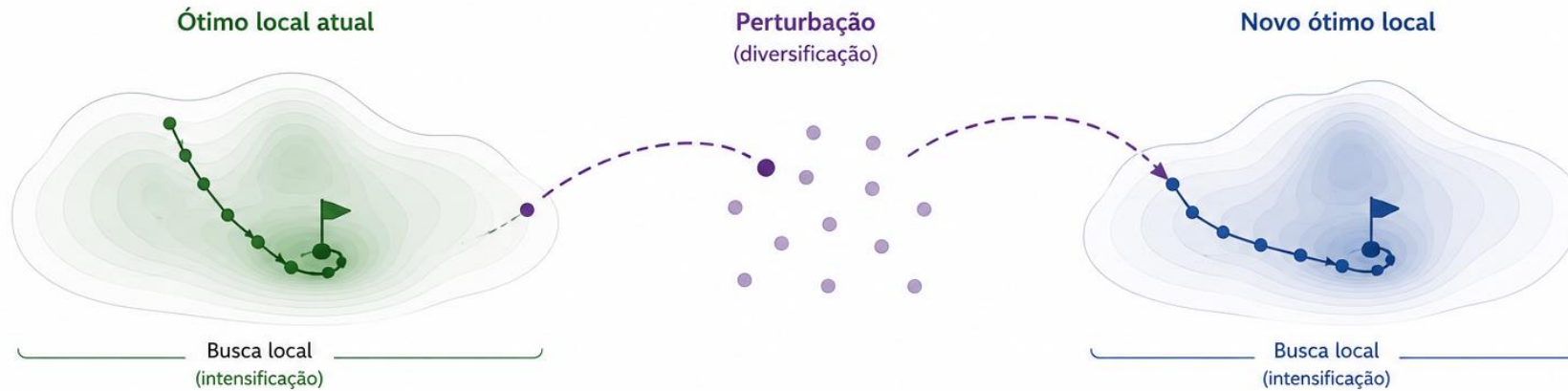
O que é perturbação?

- A **perturbação** é o componente que distingue o ILS de uma busca local simples. Ela é uma modificação deliberadamente aleatória e agressiva aplicada a uma solução de ótimo local, com o objetivo de movê-la para uma bacia de atração diferente — uma região do espaço de soluções de onde a busca local convergirá para um ótimo local diferente.
- Para que a perturbação seja eficaz, ela precisa satisfazer três propriedades simultâneas:
 - **Suficientemente agressiva:** deve modificar a solução de forma que a busca local, ao ser aplicada novamente, não retorne ao mesmo ótimo local de onde partiu. Uma perturbação fraca demais, equivalente a um movimento da busca local, simplesmente desfaz o que foi feito.
 - **Suficientemente controlada:** não pode destruir toda a estrutura da solução. Uma perturbação que embaralha a solução aleatoriamente por completo é equivalente a um reinício do zero, e desperdiça toda a informação acumulada nas iterações anteriores.
 - **Não reversível pela busca local:** idealmente, a perturbação deve criar modificações que a busca local não consegue desfazer. Essa é a propriedade mais sutil e importante



Na prática, o ponto certo de agressividade da perturbação é uma questão empírica que depende da instância e dos movimentos da busca local.

O que é perturbação? (cont.)



UMA PERTURBAÇÃO EFICAZ DEVE SATISFAZER TRÊS PROPRIEDADES

1 SUFICIENTEMENTE AGRESSIVA



Deve mover para outra bacia de atração.

⊗ Perturbação fraca = busca local retorna ao mesmo ótimo local.

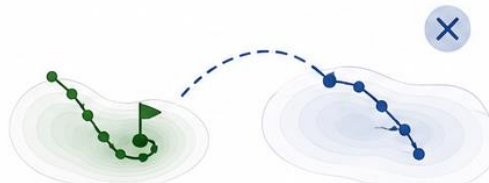
2 SUFICIENTEMENTE CONTROLADA



Não pode destruir toda a estrutura da solução.

⊗ Perturbação excessiva = equivalente a reinício do zero, desperdiça informação.

3 NÃO REVERSÍVEL PELA BUSCA LOCAL



Modificações que a busca local não consegue desfazer.

✓ Impede que a busca local volte ao mesmo ótimo local.

—●— Busca local (intensificação)

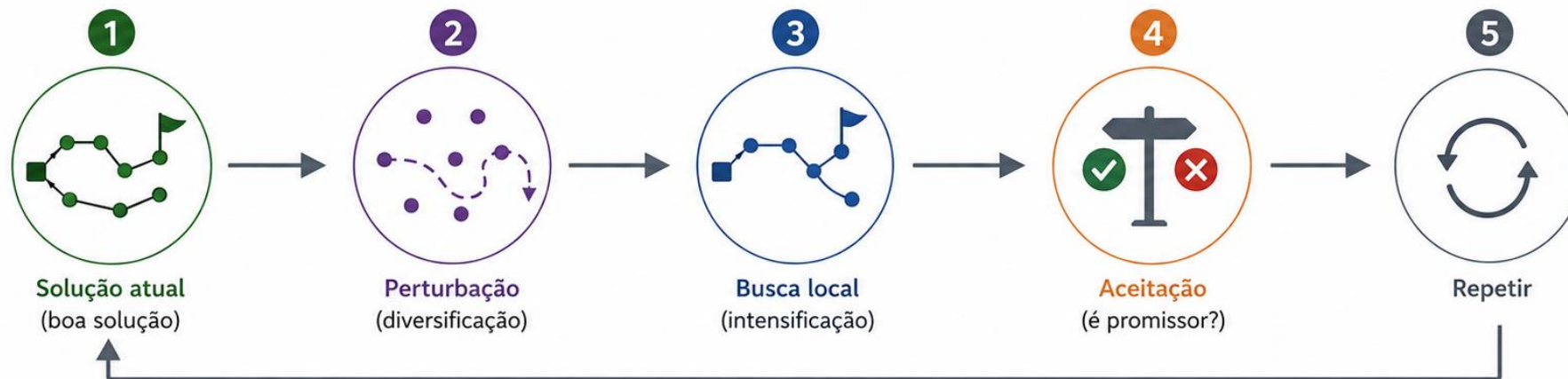
-●- Perturbação (diversificação)

● Bacia de atração (região de convergência)

Iterated Local Search (ILS)

O ILS não é uma busca aleatória. Ele é uma busca guiada por informação: cada perturbação parte de um ótimo local, não de uma solução aleatória. Isso é o que o torna muito mais eficiente do que reiniciar a busca local do zero!

- O **Iterated Local Search (ILS)** é uma metaheurística que implementa sistematicamente o ciclo: *partir de uma solução de qualidade* → *perturbá-la* → *aplicar busca local* → *decidir se o resultado é promissor* → *repetir*.
- É um dos algoritmos mais eficazes para problemas de otimização combinatória, e sua simplicidade conceitual é uma virtude real.
- O ILS foi formalizado por Lourenço, Martin e Stützle (2003), mas a ideia de iterar busca local com perturbações é ainda mais antiga. Para o VRP especificamente, variantes de ILS são frequentemente competitivas com as melhores metaheurísticas disponíveis, e muito mais simples de implementar e entender.



- **Passo 1 — Solução inicial (S_0):** O ILS começa com uma solução viável. Quanto melhor a solução inicial, maior a probabilidade de o ILS encontrar um bom ótimo global. Na Sprint 3, usamos a melhor solução da Sprint 2 (heurística construtiva + busca local) como ponto de partida. Isso não é obrigatório — mas é a escolha inteligente.
- **Passo 2 — Busca local $\rightarrow S^*$:** Antes de entrar no laço principal, aplicamos busca local completa (2-opt + Relocate) à solução inicial. Isso garante que partimos de um ótimo local bem definido, não de uma solução construtiva bruta.
- **Passo 3 — Perturbação $\rightarrow S'$:** Aplicamos uma perturbação a S^* . O resultado S' não é necessariamente melhor que S^* — na maioria das vezes, é pior. Isso é esperado e desejado: S' deve estar em uma bacia de atração diferente.
- **Passo 4 — Busca local $\rightarrow S''^*$:** Aplicamos busca local a S' . O resultado S''^* é o ótimo local da bacia de atração que a perturbação descobriu.
- **Passo 5 — Critério de aceitação:** Comparamos S''^* com S^* . Se S''^* é melhor, substituímos S^* por S''^* e voltamos ao Passo 3. Se é pior, decidimos: com critério estrito, descartamos e voltamos com S^* . Com critério de tolerância, podemos aceitar S''^* mesmo sendo ligeiramente pior, para explorar regiões novas.
- **Passo 6 — Atualização da melhor global:** Independentemente do critério de aceitação, sempre guardamos a melhor solução encontrada em qualquer iteração. Essa é a solução final retornada pelo algoritmo.

Solução inicial

- A solução inicial do ILS tem dois papéis:
 - ❖ O primeiro é óbvio: ela determina o ponto de partida da exploração.
 - ❖ O segundo é mais sutil: ela determina a **qualidade do ótimo local a partir do qual as perturbações serão aplicadas**. Um ótimo local ruim como ponto de partida significa que as primeiras perturbações explorarão regiões de baixa qualidade, e o ILS levará mais iterações para encontrar regiões boas.



- 3 opções de solução inicial para o ILS:
 - ❖ a solução do Nearest Neighbor (construção rápida, qualidade moderada);
 - ❖ a solução do Clarke-Wright (construção mais lenta, qualidade superior), ou
 - ❖ a melhor solução da Sprint 2 (busca local já aplicada — melhor ponto de partida possível com o que já temos).

Busca local no ILS: intensificação

- Cada vez que a busca local é chamada, ela leva a solução ao ótimo local mais próximo, extraindo o máximo de qualidade disponível naquela bacia de atração (intensificação);
- A escolha dos movimentos da busca local afeta diretamente a qualidade dos ótimos locais encontrados e o tempo computacional por iteração.
 - ❖ Movimentos mais fortes (2-opt + Relocate + Swap) encontram ótimos locais melhores, mas tornam cada iteração mais cara — reduzindo o número de iterações que cabem no tempo disponível.
- Há um *trade-off* real: **menos iterações com busca local mais forte, ou mais iterações com busca local mais fraca?**
 - ❖ A resposta depende da instância. Para instâncias menores (C1, C2), busca local mais forte tende a ser melhor. Para instâncias maiores (C3, C4), mais iterações com busca local mais simples pode ser mais eficaz.

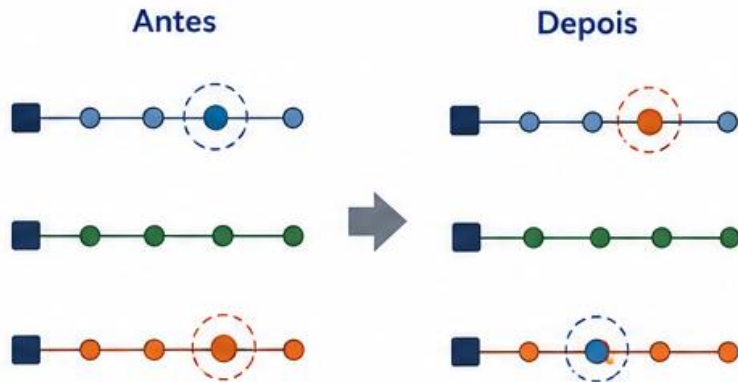
Na Sprint #3, busca local é fixada como 2-opt + Relocate para todas as equipes. A variação experimental está na perturbação e no critério de aceitação.



Perturbações P1, P2, P3

P1 RELOCATE ALEATÓRIO (k=2)

Move 2 clientes para rotas aleatórias
(viabilidade apenas)



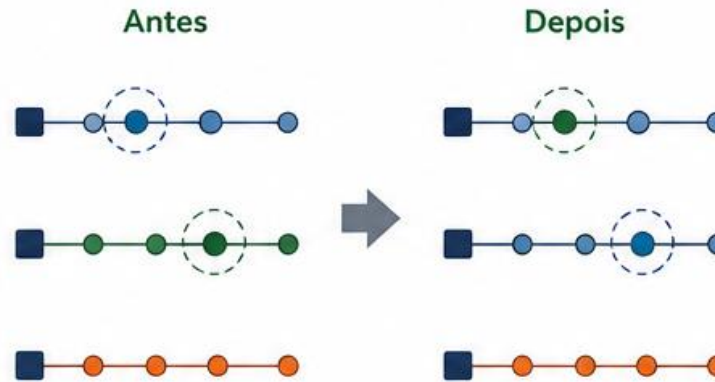
Simples
e rápida



Pode se comportar
como movimento da
busca local

P2 SWAP ALEATÓRIO (k=2)

Troca 2 pares de clientes entre
rotas distintas



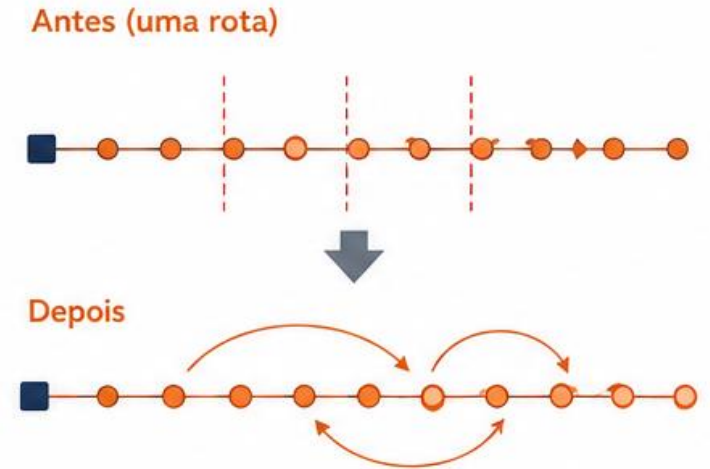
Mantém
balanceamento
das rotas



Agressividade
moderada

P3 DOUBLE-BRIDGE

Divide uma rota em 4 segmentos e
reconecta em outra ordem



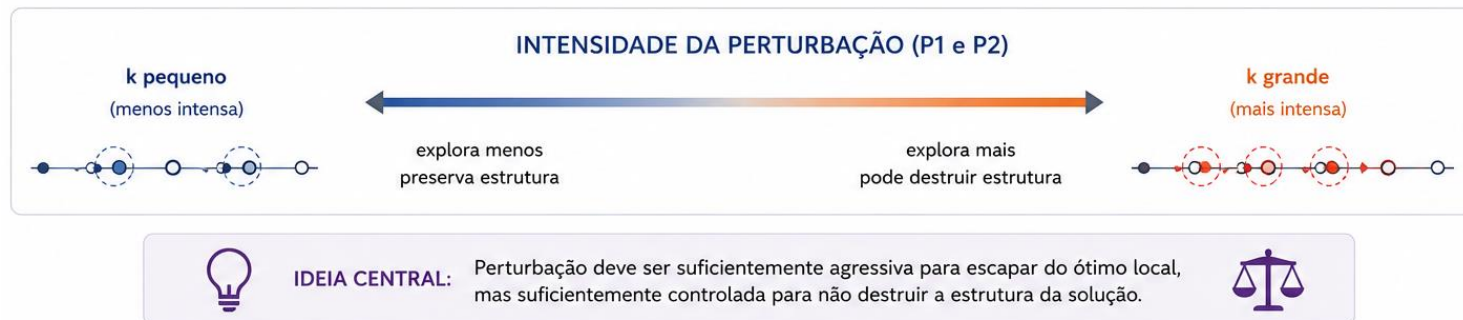
Não reversível
pelo 2-opt



Mais eficaz em
instâncias
maiores

Perturbações P1, P2, P3 (cont.)

- **P1 — Relocate aleatório ($k=2$):** Move 2 clientes escolhidos aleatoriamente para rotas aleatórias (sem critério de custo — apenas viabilidade). É simples, rápida e fácil de entender. O risco é que ela pode se comportar como um movimento da busca local (se os dois clientes escolhidos são os que deveriam ser relocados mesmo assim, a perturbação pode desfazê-los e a busca local reconverge para o mesmo ótimo local).
- **P2 — Swap aleatório ($k=2$):** Troca 2 pares de clientes entre rotas distintas, aleatoriamente. Mantém aproximadamente o mesmo número de clientes por rota, o que pode ser útil quando as rotas já estão bem balanceadas. Similar ao P1 em termos de agressividade.
- **P3 — Double-bridge:** Divide uma rota em 4 segmentos e os reconecta em uma ordem que **nenhum movimento 2-opt consegue desfazer**. Essa é a propriedade-chave: a perturbação não apenas modifica a solução, ela a move para uma bacia de atração que a busca local 2-opt não pode atingir por conta própria.
 - ❖ Para instâncias maiores, o Double-bridge é geralmente a perturbação mais eficaz precisamente por essa razão. A intensidade da perturbação (controlada por k em P1 e P2) é um parâmetro importante.



Critérios de aceitação



- O **critério de aceitação** do ILS é conceitualmente diferente do critério da busca local. A busca local aceita movimentos que melhoram imediatamente o custo. O ILS aceita *soluções* ao final de uma iteração completa (perturbação + busca local).

❖ **Aceitação estrita:** aceita S''^* apenas se $\text{custo}(S''^*) < \text{custo}(S^*)$. O algoritmo sempre trabalha a partir da melhor solução encontrada até o momento. Pode ficar preso em bacias de atração de alta qualidade relativa, sem nunca explorar regiões distantes que poderiam ser ainda melhores.

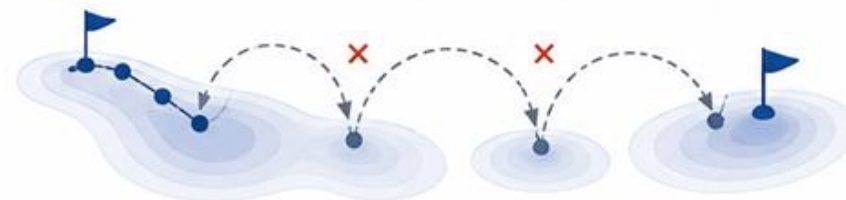
❖ **Aceitação com tolerância (δ):** aceita S''^* se $\text{custo}(S''^*) \leq \text{custo}(S^*) \times (1 + \delta)$. Com $\delta=3\%$, aceita soluções até 3% piores que a corrente. Isso permite que o ILS "suba colinas" ocasionalmente, explorando regiões do espaço que o critério estrito rejeitaria. O risco é aceitar soluções muito piores se δ for grande, degradando a qualidade da solução corrente.

- **Isso não é Simulated Annealing!** No SA, a probabilidade de aceitar soluções piores decresce ao longo do tempo seguindo um schedule de temperatura. Aqui, δ é fixo — é uma aceitação determinística com tolerância constante.



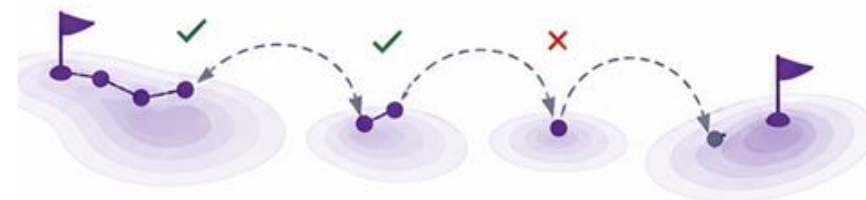
1 ACEITAÇÃO ESTRITA

Aceita S''^* apenas se
 $\text{custo}(S''^*) < \text{custo}(S^*)$



2 ACEITAÇÃO COM TOLERÂNCIA (δ)

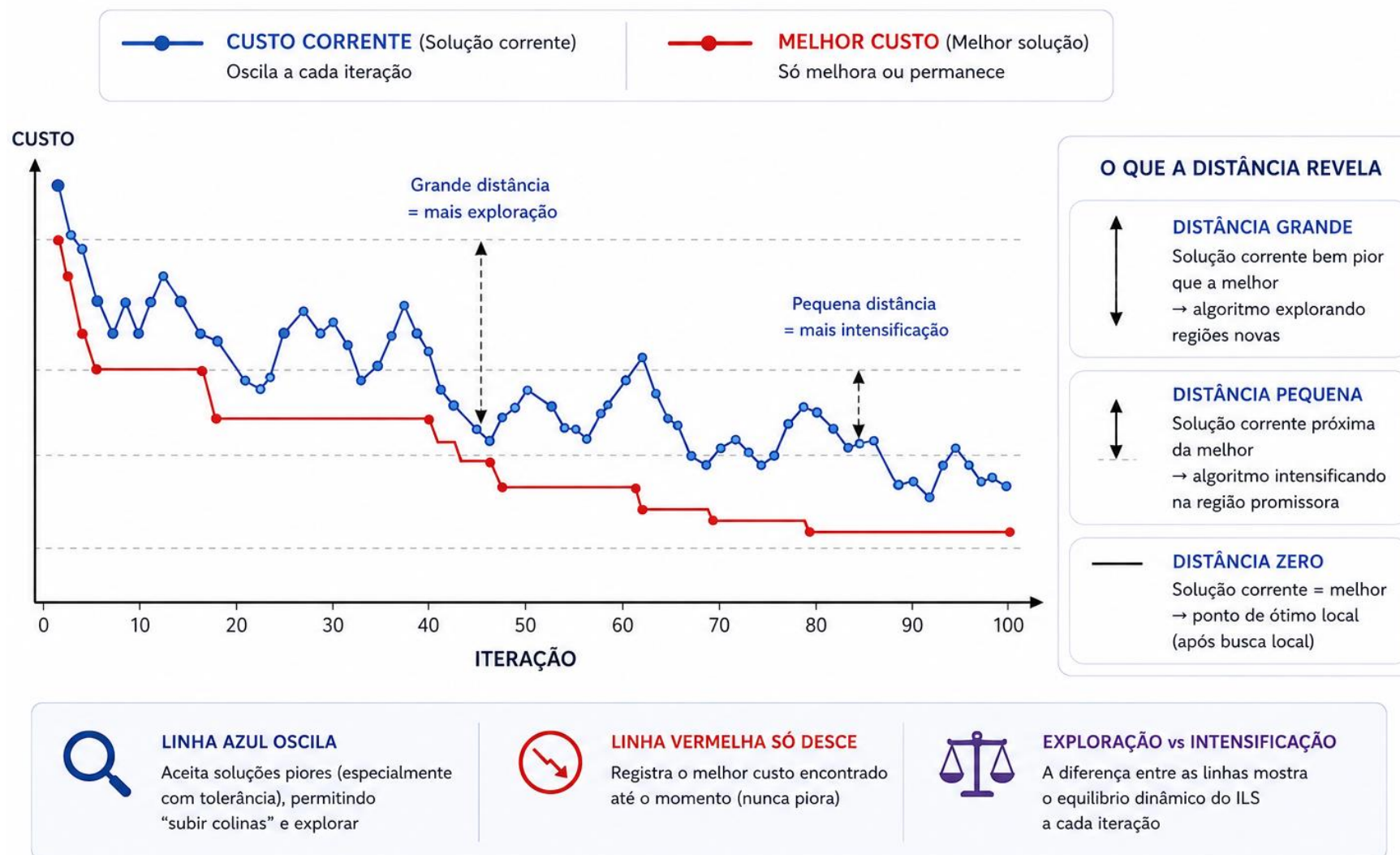
Aceita S''^* se
 $\text{custo}(S''^*) \leq \text{custo}(S^*) \times (1 + \delta)$



Atualização da melhor solução

- Um aspecto sutil do ILS que merece atenção: a distinção entre solução corrente e melhor solução global.
 - ❖ A solução corrente é a que o ILS usa como ponto de partida para a próxima perturbação.
- Com aceitação estrita, ela é sempre a melhor encontrada até agora;
- Com aceitação por tolerância, ela pode ser uma solução ligeiramente pior — aceita para explorar uma nova região.
- A melhor solução global é a melhor já encontrada em qualquer iteração, independentemente do critério de aceitação. Ela nunca piora ao longo das iterações — é uma sequência monótona decrescente. É essa solução que o ILS retorna ao final.
- A distinção é importante porque, com critério de tolerância, o ILS pode temporariamente "piorar" a solução corrente para explorar — mas nunca perde de vista a melhor já encontrada. O registro do histórico de convergência captura ambas as trajetórias.
- **Curva de convergência:** a linha azul (custo corrente) oscila, especialmente com critério de tolerância; a linha vermelha (melhor custo) só desce. A diferença entre as duas linhas revela quanto o algoritmo está explorando versus intensificando em cada iteração.

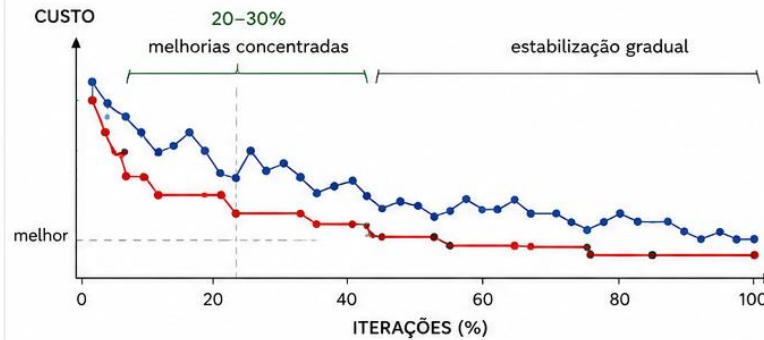
Curva de convergência



Curva de convergência (cont.)

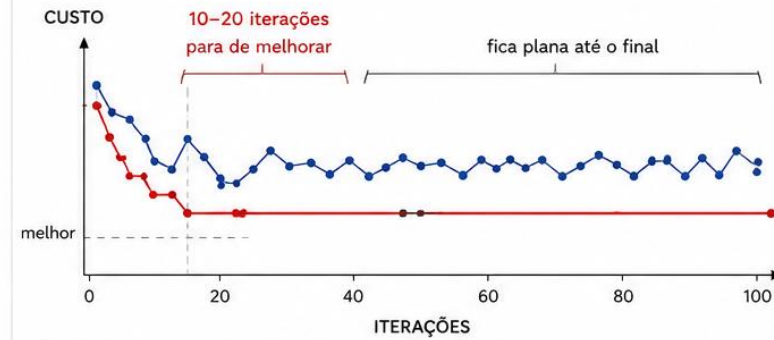
— CUSTO CORRENTE (oscila) — MELHOR CUSTO (só desce)

1 PADRÃO IDEAL



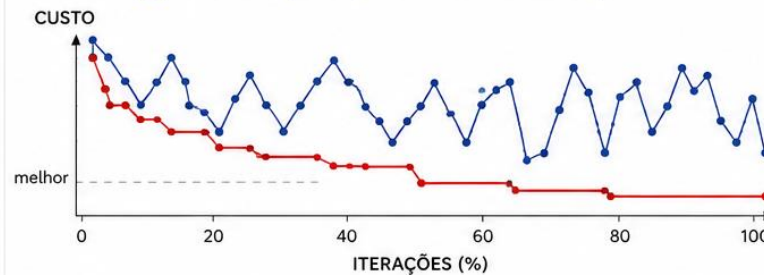
Explorou bem o espaço e convergiu para um bom ótimo.

2 CONVERGÊNCIA PREMATURA



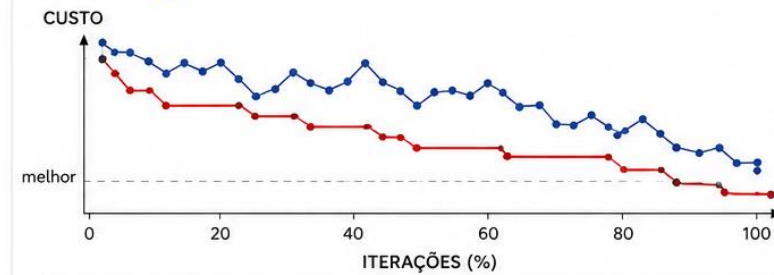
Perturbação não eficaz em escapar dos ótimos locais. Retorna sempre ao mesmo ótimo local.

3 OSCILAÇÃO (COM TOLERÂNCIA)



A tolerância está funcionando: aceita soluções piores para explorar, e eventualmente encontra regiões melhores.

4 NÃO CONVERGÊNCIA



A curva ainda está descendo nas últimas iterações.
O ILS não convergiu dentro do limite de iterações — o potencial de melhoria não foi esgotado.

Variações por equipe

- A Sprint 3 é, em sua essência, um **experimento controlado de design fatorial**. O espaço experimental tem duas dimensões: *tipo de perturbação* e *critério de aceitação*. Cada equipe ocupa um ponto específico desse espaço:

	Aceitação estrita	Tolerância $\delta = 3\%$
Relocate aleatório (k=2)	3VA – Equipe 1 3VB – Equipe 1	3VA – Equipe 3
Double-bridge	3VA – Equipe 2 3VB – Equipe 2	3VA – Equipe 4 3VB – Equipe 3



Indispensável comparar com a solução exata (ou melhor solução conhecida) da Sprint 1!

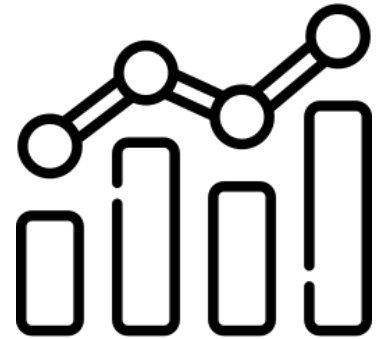
Como comparar os resultados?

- Comparar algoritmos heurísticos requer cuidado metodológico. Três erros comuns:
 - ❖ **Erro 1 — Comparar sem controlar a seed.** Dois algoritmos rodados com seeds diferentes podem produzir resultados diferentes por puro acaso estatístico. Para comparação justa, use sempre a mesma seed em todas as variantes. No notebook, seed=42 está fixada.
 - ❖ **Erro 2 — Comparar em instâncias diferentes.** Uma variante pode parecer melhor simplesmente porque foi testada em uma instância mais fácil. Para comparação justa, todas as equipes rodam as mesmas instâncias C1–C4.
 - ❖ **Erro 3 — Comparar apenas o resultado final sem analisar o processo.** O custo final é importante, mas a **curva de convergência** conta uma história muito mais rica: em quantas iterações a melhoria aconteceu? O algoritmo estabilizou cedo ou continuou melhorando até o final?
- Esses padrões revelam a eficiência do algoritmo, não apenas sua eficácia. O correto é: mesmo número de iterações, mesma instância, mesma seed, mesma solução inicial. Só então as diferenças nos resultados podem ser atribuídas às diferenças de configuração.



Métricas de desempenho

- **Custo total (R\$):** a métrica primária. Quanto menor, melhor.
 - ❖ Reportar em três momentos: solução inicial carregada, após busca local standalone (Baseline 1), após ILS. O ganho do ILS sobre o Baseline 1 é o indicador mais importante do valor adicionado pela metaheurística.
- **Número de rotas e composição da frota:** o ILS pode, via Relocate na busca local, consolidar rotas e reduzir o número de veículos necessários. Uma rota a menos significa um veículo a menos na operação, com impacto direto em custo fixo.
- **Ganho percentual por instância:** calcule $(\text{custo_baseline1} - \text{custo_ILS}) / \text{custo_baseline1} \times 100$. Analise se o ganho aumenta ou diminui com o tamanho da instância. Isso responde: *o ILS escala bem?*
- **Número de melhorias por iteração:** quantas vezes, ao longo das N iterações, o ILS encontrou uma nova melhor solução.
 - ❖ Um número alto de melhorias concentradas no início indica boa perturbação; melhorias espalhadas ao longo de todas as iterações indicam que o algoritmo ainda tinha potencial quando parou.
- **Tempo computacional:** registre separadamente o tempo da busca local inicial, de cada iteração de ILS e do total. Para recomendar o algoritmo à Prolog, o tempo é tão importante quanto a qualidade..



Parte prática – Python/Colab

